

附件 1: **计算机科学与工程学院课程设计报告**

题目全称: _____

题目难度等级: _____

指导老师: _____ 职称: _____

学生姓名: _____ 学号: _____

专业: _____ 班号 (大类分流后班号): _____

A 计算机使用技能 (100 分)	B 课程设计报告 (100 分)	C 计算机专业技能 (100 分)	总分 ($A*10\%+B*30\%+C*60\%$)

备注: 如参加答辩, 请答辩老师给出 C 计算机专业技能的分数。请指导教师给出除计算机专业技能以外的其他分数。

如参加答辩, 请答辩老师签字: _____

主要任务:

详细功能描述:

预期成果或目标:

指导老师评语:

指导教师签字: _____

基于 Web 的文件上传、下载与删除管理系统 设计与实现

项目地址: <https://design.zhaoconglei.com/>

开源代码: https://github.com/2024080911015/class_design

目录

第一章 绪论	5
1.1 项目完成情况	5
1.2 课题背景与意义	6
1.3 文件管理系统介绍	8
1.3.1 系统主要功能	8
1.3.2 系统操作方式	8
1.4 相关开发技术介绍	10
1.4.1 HTML 与 CSS 技术介绍	10
1.4.2 jQuery UI 技术介绍	10
1.4.3 Flask 框架介绍	11
第二章 系统分析	12
2.1 需求分析	12
2.1.1 文件上传需求	12
2.1.2 文件下载需求	13
2.1.3 文件删除需求	13
2.2 功能分析	14
2.2.1 基本功能	14
2.2.2 拓展功能	14
2.3 系统结构分析	15
第三章 详细设计及实现	17
3.1 系统总体设计	17
3.2 用户界面设计	18
3.2.1 文件上传区域设计	19
3.2.2 文件管理区域设计	20

3.3	jQuery UI 组件设计	21
3.3.1	选项卡组件	21
3.3.2	按钮组件	22
3.3.3	进度条组件	25
3.3.4	对话框组件	25
3.4	文件上传功能实现	26
3.4.1	单文件上传实现	27
3.4.2	文件夹上传实现	27
3.4.3	上传进度显示	28
3.5	文件下载功能实现	28
3.5.1	后端下载逻辑实现	28
3.5.2	前端下载触发机制	29
3.6	文件删除功能实现	29
3.6.1	前端删除流程	30
3.6.2	后端删除逻辑	30
3.7	异常处理与安全设计	30
3.7.1	文件路径安全防护	31
3.7.2	用户访问控制	31
3.7.3	操作异常处理	32
第四章	系统测试	33
4.1	测试环境	33
4.1.1	硬件环境	33
4.1.2	软件环境	33
4.2	页面显示测试	34
4.3	文件上传测试	35
4.4	文件下载测试	36
4.5	文件删除测试	37
4.6	测试小结	38
第五章	总结与展望	40
5.1	课程设计总结	40
5.2	系统不足	41

5.3 改进方向	41
--------------------	----

第一章 绪论

1.1 项目完成情况

本次课程设计完成了一个基于 Web 的文件管理系统，系统主要围绕文件上传、文件下载和文件删除三个核心功能展开。项目采用 Flask 框架作为后端服务，使用 HTML 和 CSS 设计前端用户界面，并结合 JavaScript 与 jQuery UI 实现页面交互效果。系统整体界面分为两个主要部分：第一部分为文件上传区域，第二部分为文件管理区域，用户可以在文件管理区域查看已上传文件，并对文件进行下载或删除操作。

在功能实现方面，系统已经完成了单文件上传和文件夹上传功能。用户可以通过页面中的文件选择区域选择本地文件，并将文件上传到服务器指定目录中。后端通过 Flask 接收上传请求，并将文件保存到 uploads 文件夹中。除了单文件上传外，系统还实现了文件夹上传功能，用户可以选择整个文件夹进行上传，系统会保留文件夹内部的目录结构，从而提高文件管理的完整性和实用性。

在文件管理方面，系统实现了文件列表显示、文件下载和文件删除功能。后端能够读取服务器端 uploads 目录下的文件和文件夹信息，并将其整理为目录结构数据返回给前端。前端页面根据返回的数据生成文件列表，显示文件名称、文件大小、修改时间等信息。用户点击下载按钮时，可以下载对应文件；如果下载的是文件夹，系统会自动将文件夹压缩为 zip 文件后再提供下载。用户点击删除按钮时，系统会弹出确认对话框，确认后可删除对应的文件或文件夹。

在交互设计方面，系统使用了 jQuery UI 组件库对页面功能进行增强。上传区域使用 Tabs 组件区分单文件上传和文件夹上传；操作按钮使用 Button 组件进行美化；上传过程中使用 Progressbar 组件显示上传状态；删除前使用 Dialog 组件弹出确认提示，避免用户误删。通过这些组件的使用，系统界面更加清晰，用户操作也更加直观。

此外，系统还实现了用户注册、登录和退出功能。用户访问系统主页时，如果未登录会跳转到注册或登录页面，登录成功后才能进入文件管理页面。系统使用 JSON 文件保存用户信息，并通过 session 记录用户登录状态。虽然用户管理不是本课程设

计题目要求的核心功能，但它可以提高系统的完整性，使文件管理系统更加接近实际 Web 应用。

在安全性和异常处理方面，系统对上传和访问路径进行了规范化处理，避免用户通过特殊路径访问 uploads 目录以外的文件。当用户上传空文件、删除不存在的文件或访问非法路径时，系统能够返回相应的错误提示。整体来看，本项目已经基本完成课程设计要求，实现了基于 HTML、CSS 和 jQuery UI 的文件管理页面，并完成了文件上传、文件下载和文件删除三个主要功能。

1.2 课题背景与意义

随着计算机网络技术和 Web 应用技术的不断发展，越来越多的日常工作开始从本地单机操作转向基于浏览器的在线操作。在学习、办公和软件开发过程中，文件的上传、下载、保存和删除是非常常见的需求。例如，学生在提交课程作业时需要上传实验报告或源代码文件；教师在发布教学资料时需要提供文件下载；在团队协作中，成员之间也经常需要通过网络系统共享文档、图片、压缩包等各种类型的文件。因此，设计并实现一个具有基本文件管理功能的 Web 系统，具有较强的实用价值和学习意义。

传统的文件管理通常依赖操作系统自带的文件资源管理器，用户只能在本地计算机上完成文件查看、复制、删除等操作。这种方式虽然简单直接，但不适合网络环境下的文件共享和远程管理。当用户需要在不同设备之间传输文件，或者希望通过浏览器统一管理服务器上的文件时，就需要借助 Web 文件管理系统。Web 文件管理系统能够通过浏览器提供统一的操作界面，用户不需要直接接触服务器文件目录，也不需要掌握复杂的命令行操作，只需要通过页面按钮即可完成文件上传、文件下载和文件删除等操作。

本课题所设计的文件管理系统正是围绕这一实际需求展开的。系统通过 HTML 和 CSS 完成用户界面设计，将页面划分为文件上传区域和文件管理区域两个主要部分。其中，文件上传区域用于选择并上传本地文件，文件管理区域用于展示服务器端已经保存的文件，并提供下载和删除操作。当前项目中的主页面已经按照这一思路进行设计，页面中包含“上传文件”和“文件列表”两个核心区域，其中上传区域又分为单个文件和整个文件夹两个选项，文件列表区域则展示文件名称、大小、修改时间以及操作按钮等信息。这种设计能够使用户较为清晰地理解系统功能，也便于后续进行功能扩展。

在前端交互方面，本课题要求使用 jQuery UI。jQuery UI 是基于 jQuery 的用户界面组件库，提供了按钮、对话框、选项卡、进度条等常用界面组件。相比于只使用

普通 HTML 表单和浏览器默认提示框, jQuery UI 可以让页面具有更好的交互效果和更统一的视觉风格。在本系统中, 上传部分使用选项卡组件区分单文件上传和文件夹上传, 按钮组件用于美化上传、刷新、下载和删除等操作按钮, 进度条组件用于显示上传过程状态, 对话框组件用于显示提示信息和删除确认信息。这些组件的使用使系统不再只是简单的静态页面, 而是具有一定交互能力的 Web 应用界面。

从课程设计的角度来看, 本课题能够综合运用前端页面设计、后端接口开发、文件系统操作和前后端交互等多方面知识。前端部分需要使用 HTML 组织页面结构, 使用 CSS 控制页面布局和样式, 并使用 JavaScript 与 jQuery UI 实现用户交互; 后端部分需要使用 Flask 框架接收浏览器请求, 完成文件保存、文件读取、文件下载和文件删除等操作; 同时还需要考虑文件路径处理、异常提示、页面刷新和用户操作反馈等问题。因此, 该课题不是单一知识点的简单练习, 而是一个具有完整功能流程的小型 Web 应用项目。

本课题的意义主要体现在以下几个方面。首先, 通过实现文件上传、下载和删除功能, 可以加深对 Web 请求与响应过程的理解。文件上传需要浏览器将本地文件通过表单或 Ajax 请求提交给服务器, 服务器接收后再保存到指定目录; 文件下载则需要服务器将指定文件作为响应内容返回给浏览器; 文件删除则涉及前端确认、后端定位文件以及执行删除操作等步骤。通过这些功能的实现, 可以更加直观地理解浏览器与服务器之间的数据交互过程。

其次, 本课题有助于提高前端界面设计能力。一个可用的系统不仅要能完成后台功能, 还需要有清晰、友好的用户界面。通过本系统的设计, 可以学习如何将页面划分为不同功能区域, 如何设计上传框、文件列表、操作按钮和提示对话框, 如何使用 CSS 控制布局、间距、颜色和响应式效果。同时, jQuery UI 组件的使用也可以帮助理解前端组件化思想, 即通过封装好的组件提高开发效率和界面一致性。

再次, 本课题能够锻炼后端开发和文件操作能力。文件管理系统虽然规模不大, 但涉及的后端逻辑比较完整, 包括接收上传文件、创建保存目录、遍历服务器文件夹、返回文件列表数据、判断文件或文件夹类型、压缩文件夹、删除文件或目录等内容。这些操作都与实际 Web 系统中的资源管理功能密切相关。通过本课题的实现, 可以进一步理解 Flask 路由、请求对象、响应对象以及服务器端文件系统操作的基本方法。

最后, 本课题还具有一定的实际应用价值。虽然本系统属于课程设计项目, 功能规模相对较小, 但它已经具备了一个基础文件管理平台的主要功能。在此基础上, 后续可以继续扩展用户权限管理、文件分类管理、文件预览、文件搜索、上传进度显示、文件大小限制、数据库存储和多用户隔离等功能, 使系统更加接近真实场景中的在线

文件管理平台。因此，本课题不仅能够满足课程设计对 HTML、CSS、jQuery UI 和文件操作功能的要求，也为后续学习更复杂的 Web 应用开发打下基础。

1.3 文件管理系统介绍

1.3.1 系统主要功能

本系统是一个基于 Web 的文件管理工具，主要围绕文件的日常管理需求进行设计，提供文件上传、文件下载和文件删除三个核心功能。

在文件上传方面，系统支持单文件上传和整个文件夹上传两种方式。单文件上传允许用户选择本地计算机中的任意一个文件并将其上传至服务器的指定存储目录中；文件夹上传则允许用户选择整个文件夹，系统会自动读取文件夹内的所有文件及其相对路径，并将目录结构完整地保存到服务器端。这两种上传方式可以满足不同场景下的文件传输需求。

在文件下载方面，系统支持单个文件下载和文件夹下载两种形式。对于单个文件，用户点击下载按钮后，浏览器会直接从服务器获取文件并保存到本地。对于文件夹，系统会先将文件夹中的所有文件打包成一个 zip 压缩包，再提供给用户下载，从而保证文件夹内的目录结构不会丢失。

在文件删除方面，系统提供文件删除和文件夹删除功能。用户在查看文件列表时，可以对不需要的文件或文件夹执行删除操作。为防止用户误操作，系统在执行删除之前会弹出确认对话框，用户确认后方可完成删除。删除操作会从服务器存储目录中彻底移除对应的文件或文件夹。

除了上述三个核心功能外，系统还实现了用户注册、登录和退出功能。用户首次使用时需要注册一个账号，之后通过登录进入系统主页。退出功能使用户可以安全地结束当前会话。用户管理虽然不是本系统的核心功能要求，但它为系统提供了基本的访问控制能力，使不同用户的使用环境相互隔离。

1.3.2 系统操作方式

用户通过浏览器访问系统主页。如果用户尚未登录，系统会自动跳转到注册或登录页面。新用户需要填写用户名和密码完成注册，注册成功后跳转到登录页面，输入正确的账号和密码后方可进入文件管理主页面。

系统主页面以左右两栏布局呈现。页面左侧为文件上传区域，右侧为文件管理区域，两个区域在功能上相互独立，便于用户同时进行上传和管理操作。

在文件上传区域中，系统使用选项卡组件将上传方式划分为单个文件上传和整个文件夹上传两个标签页。默认显示的是单文件上传标签页，用户可以通过点击选项卡切换到文件夹上传标签页。每个标签页内都有一个文件选择区域，用户点击该区域或拖拽文件到区域中即可选择要上传的内容。在选择文件后，页面上会显示已选择的文件名称或文件夹中的文件数量，确认无误后点击上传按钮即可将文件发送至服务器。上传过程中，进度条会实时显示上传进度。

在文件管理区域中，系统以列表形式展示服务器端存储的所有文件和文件夹。列表中的每一行代表一个文件或文件夹，依次显示名称、大小和修改时间。对于文件夹，用户可以通过点击名称前的箭头图标展开或折叠文件夹，查看其中的子文件和子文件夹。每个文件或文件夹右侧都对应有下载和删除两个操作按钮。用户点击下载按钮即可将文件保存到本地；点击删除按钮会弹出确认对话框，用户确认后才会执行删除操作。文件列表还提供了刷新按钮，用户可以随时刷新列表以获取最新的文件状态。系统的主页面布局如图 1.1 所示。

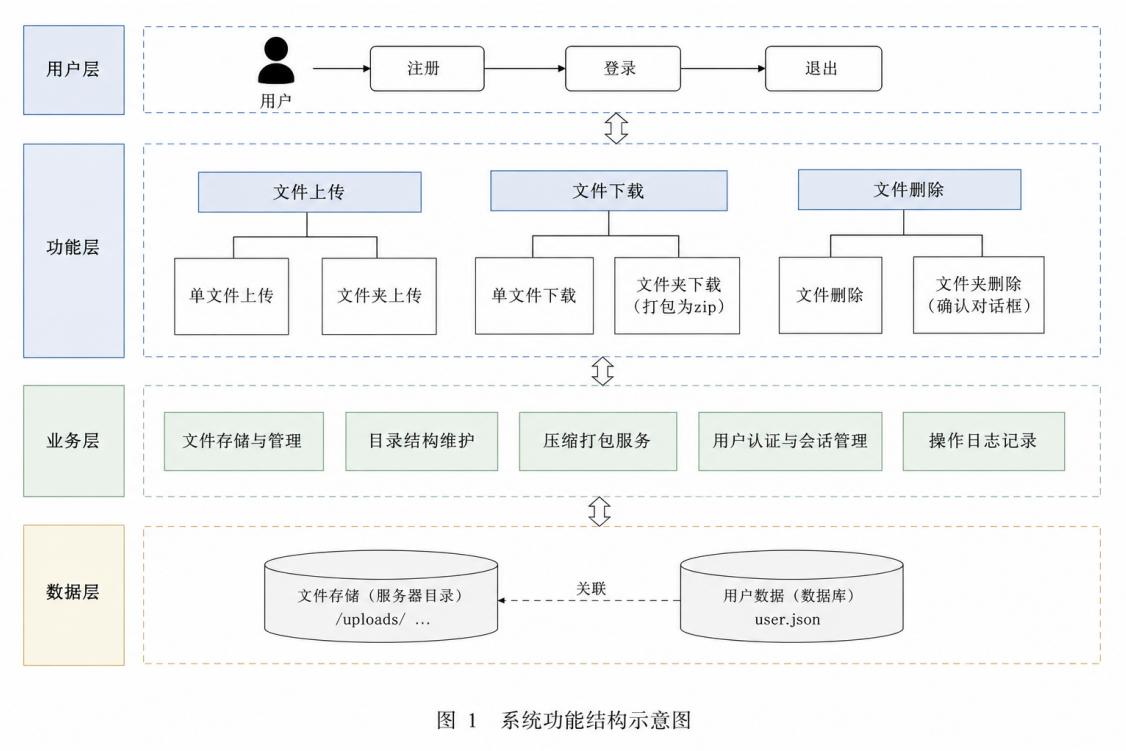


图 1 系统功能结构示意图

图 1.1: 文件管理系统主页面

1.4 相关开发技术介绍

1.4.1 HTML 与 CSS 技术介绍

HTML (HyperText Markup Language, 超文本标记语言) 是构建 Web 页面的基础语言。它通过一系列的标签 (Tag) 来定义网页的结构和内容, 包括标题、段落、表格、表单、图片和链接等元素。在文件管理系统中, HTML 负责搭建页面的基本骨架。例如, `<header>` 标签定义了页面顶部栏, `<section>` 标签将页面划分为上传区域和管理区域, `<form>` 标签用于构建文件上传表单, `<input type="file">` 提供了文件选择功能。合理的 HTML 结构是页面可读性和可维护性的基础。

CSS (Cascading Style Sheets, 层叠样式表) 用于控制 HTML 元素的视觉呈现, 包括布局、颜色、字体、间距和动画效果。在本系统中, CSS 主要用于以下几个方面: 一是页面布局控制, 使用 Flexbox 和 Grid 实现顶部栏与左右两栏的版面设计; 二是组件样式美化, 为上传区域、文件列表、按钮和对话框等元素设计统一的视觉风格; 三是响应式适配, 通过媒体查询 (Media Query) 使页面在不同屏幕尺寸下都能正常显示。系统采用以 CSS 变量 (Custom Properties) 管理的主题配色方案, 通过统一修改 CSS 变量的值即可调整整体色彩风格, 便于后续的界面定制。

1.4.2 jQuery UI 技术介绍

jQuery UI 是一套基于 jQuery 构建的用户界面交互组件库, 提供了选项卡 (Tabs)、按钮 (Button)、进度条 (Progressbar)、对话框 (Dialog)、日期选择器 (Datepicker) 等一系列封装完善的界面组件。开发者无需从零编写复杂的交互逻辑, 只需调用简单的 jQuery UI API 即可快速构建具有统一视觉风格和良好交互体验的页面功能。

在本系统中, jQuery UI 的四个核心组件被用于增强文件管理页面的交互效果。选项卡组件 (Tabs) 用于上传区域, 将单文件上传和文件夹上传两个功能放在不同的标签页中, 用户点击标签即可切换上传方式, 页面布局更加紧凑清晰。按钮组件 (Button) 用于美化页面中所有的操作按钮, 包括上传、刷新、下载和删除等按钮, 使其具有统一的悬停、点击和禁用状态样式。进度条组件 (Progressbar) 用于显示文件上传过程中的进度状态, 用户在上传文件时可以实时了解传输的完成比例, 避免因等待而产生不确定感。对话框组件 (Dialog) 用于显示系统提示信息和删除确认弹窗, 提示信息对话框在操作成功或失败时向用户反馈结果, 删除确认对话框则在用户执行删除操作前弹出确认提示, 防止用户误删重要文件。

通过上述 jQuery UI 组件的使用, 系统的用户界面在功能性和易用性方面都得到

了提升。组件化的开发方式也使得前端代码的结构更加清晰，便于后续的功能扩展和维护。

1.4.3 Flask 框架介绍

Flask 是一个轻量级的 Python Web 框架，以其简洁灵活的设计理念而广受欢迎。Flask 提供了路由（Routing）、请求处理（Request Handling）、模板渲染（Template Rendering）等 Web 开发所需的核心功能，同时保持了高度的可扩展性，开发者可以根据项目需要灵活选择数据库、表单验证和用户认证等第三方扩展库。

在本系统中，Flask 作为后端服务框架承担了以下关键职责。第一，路由分发：系统通过 `@app.route()` 装饰器定义了十余个 URL 路由，包括主页路由、文件上传路由、文件下载路由、文件删除路由、文件列表路由以及用户注册、登录和退出路由，每个路由对应一个特定的后端处理函数。第二，请求处理：Flask 的 `request` 对象用于获取前端上传的文件数据和表单提交的用户信息，`send_file` 函数用于将服务器文件作为响应内容发送给浏览器端下载。第三，会话管理：Flask 内置的 `session` 对象用于记录用户的登录状态，用户在登录成功后，其用户名会被写入 `session` 中，后续请求可以通过检查 `session` 来判断用户是否已经登录，从而实现基本的访问控制。

Flask 的轻量特性使其非常适合本系统的开发需求。系统不需要复杂的数据库配置和 ORM 映射，文件直接存储在服务器的文件系统中，用户数据以 JSON 格式保存。这种简洁的技术选型降低了系统的开发和维护成本，使开发重心可以更多地放在前端界面设计和前后端交互功能上。

第二章 系统分析

2.1 需求分析

本系统面向需要通过网络浏览器进行文件管理的普通用户，围绕文件上传、文件下载和文件删除三个核心功能展开需求分析。系统不需要复杂的数据库支持和多用户权限管理体系，但要求界面清晰、操作直观，并具备基本的异常处理能力。

2.1.1 文件上传需求

文件上传是文件管理系统的基本入口，用户需要通过浏览器将本地计算机中的文件传输到服务器端保存。本系统的文件上传需求包括以下几个方面。

第一，单文件上传功能。用户应能够通过页面上的文件选择区域选取本地计算机中的任意一个文件，确认后将其上传至服务器。系统需要在前端提供清晰的文件选择入口，并在用户选定文件后显示文件名称，以使用户确认所选文件是否正确。上传完成后，系统应向前端返回上传结果信息，包括成功或失败的提示。

第二，文件夹上传功能。用户应能够选择本地计算机中的整个文件夹，系统在接收到文件夹中的所有文件后，需要在服务器端保留原有的目录结构。例如，用户选择的文件夹中包含子文件夹和多个文件，上传后这些子文件夹和文件应按照原有的层级关系保存到服务器的存储目录中，而不是将所有文件平铺在同一目录下。这一需求在实际使用中非常常见，因为用户往往需要上传包含多个文件的目录结构。

第三，上传过程中的状态反馈。由于文件传输需要一定的时间，尤其当文件较大或网络状况不佳时，用户需要能够了解上传的进度。系统应在上传过程中实时显示进度信息，让用户知道传输是否正在进行以及已经完成了多少，避免用户因无法判断传输状态而重复操作或提前关闭页面。

第四，上传异常处理。系统需要对上传过程中可能出现的异常情况进行处理，包括用户未选择文件时点击上传按钮、上传的文件内容为空等情况。当这些异常发生时，系统应给出明确的提示信息，引导用户正确操作。

2.1.2 文件下载需求

文件下载是用户从服务器获取已保存文件的主要方式。本系统的文件下载需求包括以下几个方面。

第一，单个文件下载功能。用户应能够通过文件列表中的下载按钮，将服务器端保存的单个文件下载到本地计算机。下载时，浏览器应直接触发文件保存操作，文件的名称和格式应与服务器端保存的内容一致。对于常见的文档、图片、压缩包等文件类型，下载后的文件应当能够正常打开和使用。

第二，文件夹下载功能。当用户需要下载服务器端保存的整个文件夹时，系统应能够将文件夹内的所有文件和子文件夹打包成一个压缩包文件（zip 格式），再提供给用户下载。这一需求基于以下实际考虑：文件夹本身无法作为单一文件流直接传输，且如果不经打包处理直接逐个下载文件夹内的文件，用户需要多次操作，极其不便。通过将文件夹打包为 zip 压缩包，用户只需一次下载即可获取完整的目录结构。

第三，下载操作的简洁性。下载操作应尽可能简洁直接，用户在点击下载按钮后应立即触发下载，不需要额外的确认或填写步骤。系统应在后台处理好文件的定位、读取和传输工作，用户只需关注下载结果即可。

2.1.3 文件删除需求

文件删除使用户能够清理服务器上不再需要的文件，避免存储空间的浪费。本系统的文件删除需求包括以下几个方面。

第一，文件删除功能。用户应能够通过文件列表中的删除按钮，删除服务器上指定的单个文件。删除操作执行后，该文件应从服务器的存储目录中彻底移除，同时文件列表应即时更新，不再显示已被删除的文件。

第二，文件夹删除功能。用户应能够删除服务器端保存的整个文件夹及其内部的全部内容。当删除文件夹时，系统需要递归地移除文件夹内的所有文件和子文件夹，确保存储空间被完整释放。

第三，防误删确认机制。由于删除操作是不可逆的，用户一旦误删文件将无法恢复，因此系统在执行删除操作之前，需要弹出确认对话框，明确告知用户将要删除的文件或文件夹名称，由用户再次确认后才会执行真正的删除操作。这一机制可以有效地防止因用户误操作导致的文件丢失。

第四，删除结果反馈。删除操作执行完毕后，系统应向用户返回操作结果，告知删除是否成功。如果删除的文件或文件夹不存在，系统也应当给出相应的提示信息，而不是直接报错。

2.2 功能分析

根据上述需求分析，系统的功能可以划分为基本功能和拓展功能两大类。基本功能是实现文件管理核心流程所必需的功能模块，拓展功能则是在满足核心需求的基础上，为提升系统的完整性、安全性和易用性而增加的辅助功能。

2.2.1 基本功能

基本功能围绕文件的上传、下载和删除三个核心操作展开，是系统最核心的功能模块。

在文件上传方面，系统提供单文件上传和文件夹上传两种方式。单文件上传通过 HTML 表单的文件输入控件实现，用户点击或拖拽文件到指定区域后选择文件，提交表单后文件通过 HTTP POST 请求发送到后端。文件夹上传在单文件上传的基础上，利用 HTML5 提供的 `webkitdirectory` 属性，使用户能够选择整个文件夹，系统遍历文件夹内的所有文件并逐个上传至服务器，同时保留文件在文件夹中的相对路径，从而在服务器端还原完整的目录结构。

在文件下载方面，系统在文件列表中为每个文件和文件夹提供下载按钮。下载单个文件时，后端直接读取文件内容并通过 HTTP 响应返回给浏览器，浏览器自动触发文件保存。下载文件夹时，后端先使用 Python 的 `zipfile` 模块将文件夹内的全部文件递归地打包成一个 zip 压缩包，再将压缩包以文件流的形式返回给浏览器端下载。

在文件删除方面，系统在文件列表中为每个文件和文件夹提供删除按钮。用户点击删除按钮后，系统首先弹出确认对话框，用户在对话框中确认删除操作后，前端向后端发送删除请求。后端接收到请求后，根据请求中的路径信息定位到目标文件或文件夹：如果是文件，则调用 `os.remove()` 删除；如果是文件夹，则调用 `shutil.rmtree()` 递归删除文件夹及其全部内容。删除完成后，后端向前端返回删除结果，前端更新文件列表。

2.2.2 拓展功能

拓展功能是在基本功能之上增加的辅助功能模块，虽然不是核心操作的必需环节，但对提升系统的完整性、安全性和用户体验具有重要作用。

第一，用户注册与登录功能。系统实现了基本的用户管理功能，包括用户注册、登录和退出。用户首次使用系统时需要注册账号，注册信息保存在服务器的 JSON 文件中。后续登录时，系统验证用户输入的账号和密码与保存的信息是否一致，验证通过

后通过 Flask 的 session 机制记录用户的登录状态。用户退出时，系统清除 session 中的登录信息。这一功能为系统提供了最基本的访问控制能力，防止未授权的用户随意操作服务器上的文件。

第二，文件列表刷新功能。用户在完成上传或删除操作后，文件列表会自动刷新以展示最新的文件状态。此外，页面还提供了手动刷新按钮，用户可以随时点击刷新按钮重新加载文件列表，以便在多人操作或文件状态不确定时获取最新的文件信息。

第三，安全防护功能。系统对用户请求中的文件路径进行了严格的安全校验，通过路径规范化处理和路径穿越攻击防护，确保用户无法通过构造特殊路径来访问或操作 uploads 目录以外的文件。具体来说，系统会对上传和访问路径进行标准化处理，移除多余的路径分隔符和相对路径标记，并在访问文件前验证目标路径是否位于指定的 uploads 目录范围内。

第四，交互增强功能。系统使用 jQuery UI 组件库对页面交互进行了增强，包括使用选项卡组件组织上传方式、使用按钮组件美化操作按钮、使用进度条组件显示上传状态、使用对话框组件显示提示信息和删除确认提示。这些功能虽然不是文件管理的核心操作，但它们显著改善了用户的操作体验，使系统的界面更加友好和专业。

2.3 系统结构分析

本系统采用浏览器/服务器（B/S）架构，分为前端浏览器层和后端服务器层两层结构。前端负责用户界面展示与交互操作，后端负责业务逻辑处理与数据存储，两层之间通过 HTTP 协议进行通信。系统的整体结构如图 2.1 所示。

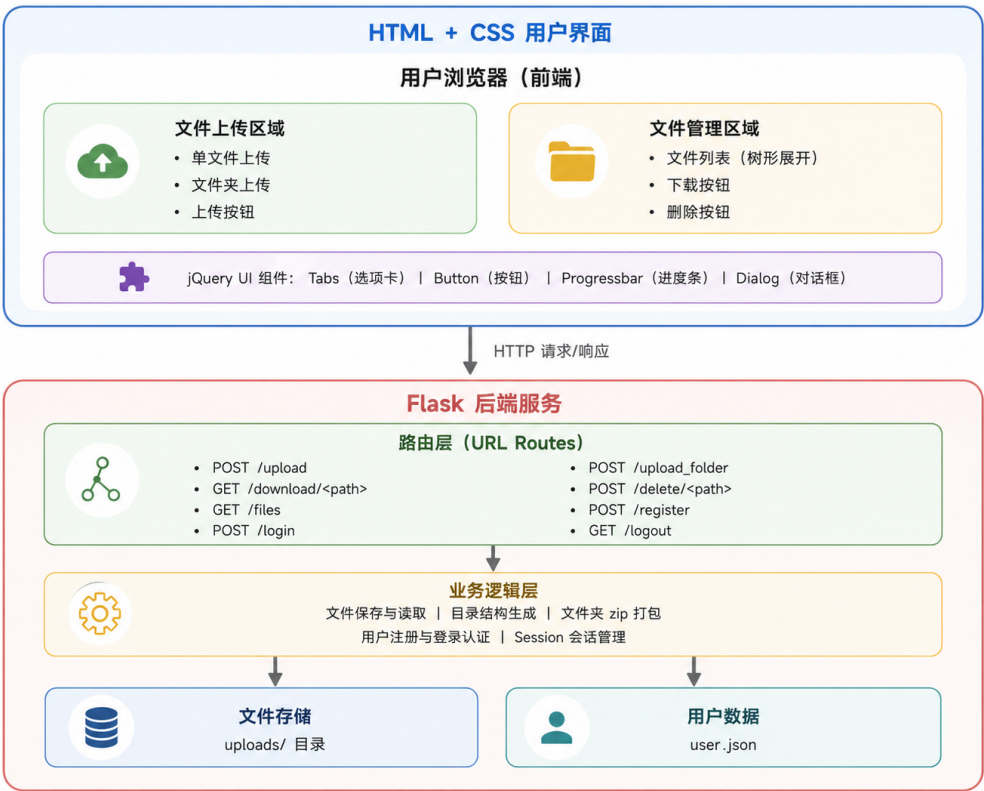


图 2.1: 系统整体结构图

前端运行在用户浏览器中，由 HTML 和 CSS 构建用户界面。页面主要划分为两个功能区域：文件上传区域和文件管理区域。文件上传区域提供单文件上传和文件夹上传两种方式，用户可通过选项卡切换。文件管理区域以列表形式展示服务器端的文件与文件夹信息，并提供下载和删除操作按钮。前端交互功能基于 jQuery UI 组件库实现，包括选项卡（Tabs）、按钮（Button）、进度条（Progressbar）和对话框（Dialog）等组件，以增强用户操作体验。

后端基于 Flask 框架实现，负责接收前端请求并执行相应的业务逻辑。路由层定义了系统提供的全部 URL 接口，包括文件上传、文件夹上传、文件下载、文件删除、文件列表获取以及用户注册、登录和退出等路由。业务逻辑层处理文件保存与读取、路径安全校验、文件夹压缩打包、用户认证和会话管理等核心逻辑。数据存储层包含两个部分：文件存储使用服务器文件系统中的 uploads 目录保存所有上传的文件和文件夹；用户数据使用 JSON 文件保存注册用户的账号信息。系统通过分层设计实现了前后端分离，各层职责清晰，便于后续功能的扩展与维护。

第三章 详细设计及实现

3.1 系统总体设计

系统总体设计从全局角度规划系统的架构层次、模块划分和交互流程，为后续的详细设计奠定基础。本系统采用浏览器/服务器（B/S）架构，整体分为前端展示层、后端业务层和数据存储层三个层次。

在前端展示层，系统使用 HTML 和 CSS 构建用户界面，并通过 JavaScript 与 jQuery UI 实现页面交互。用户首先访问的是注册和登录页面，如图 3.1 和图 3.2 所示，用户需要先注册账号，然后登录才能进入系统主页面。系统主页面分为文件上传和文件管理两个核心区域，上传区域提供单文件上传和文件夹上传两种方式，管理区域以列表形式展示文件和文件夹并提供下载与删除操作。

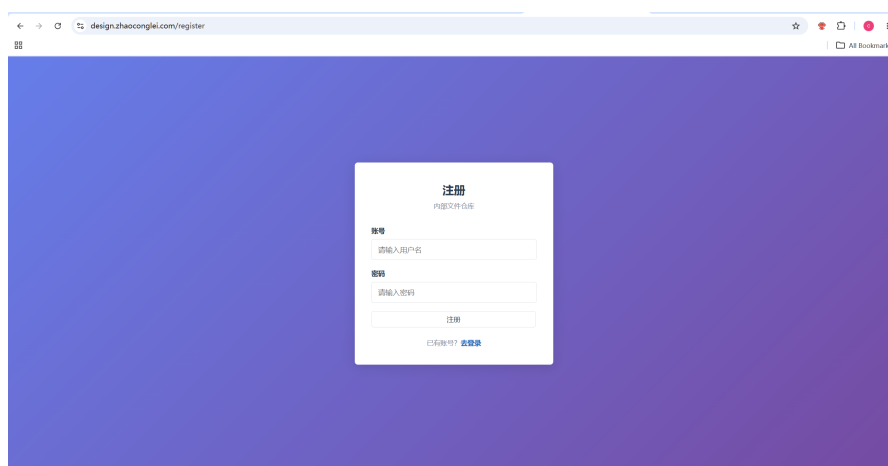


图 3.1: 用户注册页面

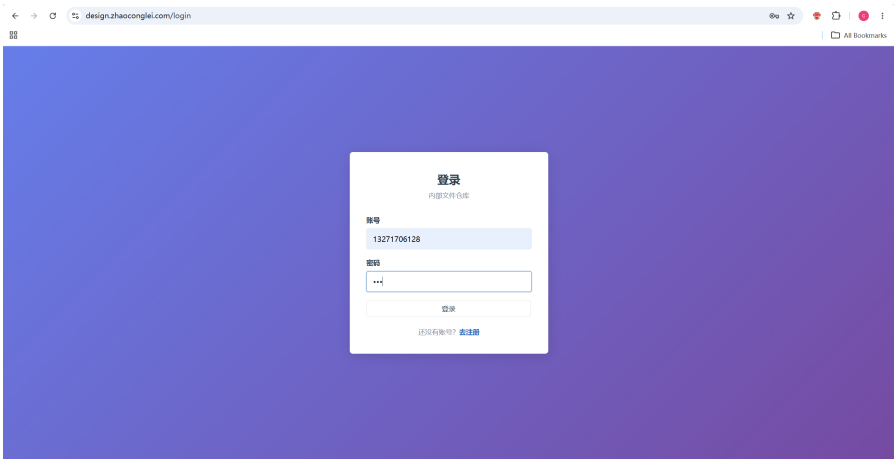


图 3.2: 用户登录页面

在后端业务层，Flask 框架负责接收前端发送的 HTTP 请求，根据路由映射关系调用对应的处理函数，执行文件保存、文件读取、文件删除、路径校验、文件夹压缩打包以及用户认证等业务逻辑。后端与前端之间通过 Ajax 异步请求和表单提交两种方式进行数据交互，交互数据格式包括 JSON 和文件流。

在数据存储层，系统采用文件系统作为存储方案。所有用户上传的文件保存在服务器上的 uploads 目录中，用户注册信息保存在 user.json 文件中。文件存储按照用户上传时的原始目录结构进行组织，便于管理和查找。这种存储方案无需配置数据库，降低了系统的部署和维护成本。

在用户操作流程方面，系统的整体交互流程如下：用户打开系统主页时，如果未登录，系统自动跳转到注册页面。新用户填写用户名和密码完成注册，注册成功后跳转到登录页面。已注册用户直接输入账号和密码进行登录，验证通过后进入文件管理主页面。在主页面中，用户可以选择文件上传到服务器，或浏览已上传的文件列表，对文件执行下载或删除操作。整个操作流程清晰连贯，用户不需要额外的学习即可快速上手使用。

3.2 用户界面设计

系统的用户界面采用 HTML 和 CSS 进行设计，整体风格简洁清晰，功能分区明确。页面主体分为文件上传区域和文件管理区域两个部分，两个区域在布局上左右并列，便于用户同时进行文件上传和文件管理操作。系统的用户界面设计如图 1.1 所示（见第 9 页）。

3.2.1 文件上传区域设计

文件上传区域位于页面左侧，是用户将本地文件传输到服务器的主要操作区域。该区域的设计充分考虑了单文件上传和文件夹上传两种场景的需求。

在布局方面，上传区域顶部设有标题”上传文件”，下方使用 jQuery UI 的选项卡组件（Tabs）将上传方式分为两个标签页。第一个标签页为”单个文件”，用于选择并上传单个文件；第二个标签页为”整个文件夹”，用于选择并上传整个文件夹及其内部的目录结构。两个标签页之间通过选项卡进行切换，默认显示单文件上传标签页，用户点击”整个文件夹”选项卡即可切换到文件夹上传模式。



图 3.3: 文件上传区域

在单文件上传标签页中，页面提供了一个虚线边框的文件选择区域（Dropzone），用户点击该区域即可打开系统文件选择对话框，选取一个文件后该区域下方会显示已选文件的名称。确认文件无误后，点击”上传文件”按钮即可将文件发送至服务器。上传过程中，页面顶部会显示进度条，实时反映上传的完成比例。

在文件夹上传标签页中，文件选择区域支持选择整个文件夹。用户点击选择区域后，系统文件选择对话框允许用户选取一个文件夹，选定后页面上会显示文件夹名称以及该文件夹中包含的文件总数。点击”上传文件夹”按钮后，系统会遍历文件夹内的

所有子文件和子文件夹，将其逐一上传到服务器，并在服务器端按照原有的目录结构进行保存。这意味着用户上传一个包含多层子目录的文件夹后，在服务器的文件列表中可以看到与本地完全一致的目录层次结构。

文件上传区域的设计以简洁和易用为原则，用户无需了解后端存储细节，只需通过选择文件和点击按钮两个步骤即可完成上传操作。选项卡的设计使两种上传方式互不干扰，用户可以按需切换，操作路径清晰明确。

3.2.2 文件管理区域设计

文件管理区域位于页面右侧，是用户浏览、管理和维护已上传文件的核心区域。该区域以列表形式展示服务器端存储的全部文件和文件夹信息，并为每个文件和文件夹提供对应的下载和删除操作入口。

在布局方面，管理区域顶部设有标题“文件列表”和刷新按钮，右侧显示文件总数统计。刷新按钮允许用户手动重新加载文件列表，以便在多人操作场景下获取最新的文件状态。列表的顶部是表头行，依次显示“名称”、“大小”和“修改时间”三列，表头右侧为“操作”列，便于用户快速定位到各列信息。

在列表主体中，每个文件或文件夹以行（Row）的形式展示。文件行和文件夹行在显示上有所不同：文件夹行左侧带有一个三角箭头图标（Collapse Toggle），用于展开或折叠文件夹以查看其内部内容；文件行则无此图标。文件夹名称左侧使用黄色图标标识“F”，文件名称左侧使用蓝色图标标识“f”，使用户可以快速区分列表中的类型。对于包含子文件和子文件夹的文件夹，用户点击箭头图标后，文件夹下方会展开显示出内部的子文件和子文件夹，且子元素相对于父文件夹有一定的缩进，从而清晰地体现出目录的层级关系。

在操作方面，每个文件行的右侧都包含“下载”和“删除”两个按钮。下载按钮用于将文件保存到本地，对于单个文件直接下载原文件，对于文件夹则自动打包为 zip 压缩包后再下载。删除按钮用于从服务器移除文件，点击后会弹出确认对话框，用户确认后才执行删除，防止误操作。文件管理区域的整体界面如图 3.4 所示。

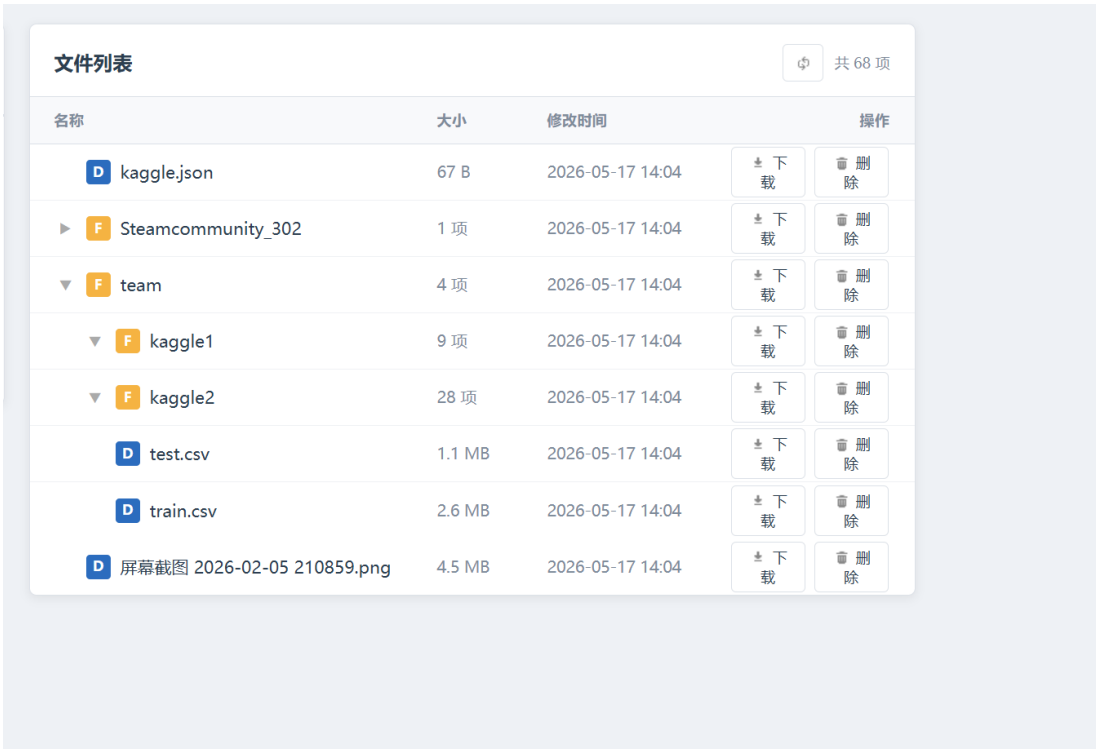


图 3.4: 文件管理区域

3.3 jQuery UI 组件设计

jQuery UI 是一套基于 jQuery 构建的用户界面交互组件库，提供了选项卡、按钮、进度条、对话框等多种常用的界面组件。在本系统中，主要使用了其中的四个组件来增强文件管理页面的交互效果和用户体验。下面分别对这些组件的设计和使用进行介绍。

3.3.1 选项卡组件

选项卡组件（Tabs）是 jQuery UI 中用于组织分类内容的重要组件，它可以将相关但不同的内容分组放置在不同的标签页中，用户通过点击标签进行切换，从而在有限的页面空间内展示更多信息。

在本系统中，选项卡组件用于文件上传区域，将单文件上传和文件夹上传两种上传方式分别放置在不同的标签页中。页面加载时，jQuery UI 通过调用 `$("#uploadTabs").tabs()` 方法对上传区域中的选项卡进行初始化。该方法会自动扫描 HTML 结构中定义的选项卡列表和对应的内容面板，将 `<ul>` 列表项中的链接与对应 `<div>` 内容面板关联起来，并添加必要的 CSS 类名和 ARIA 无障碍属性。

在交互效果方面，选项卡组件默认显示第一个标签页（单文件上传）。当用户点击”整个文件夹”选项卡时，组件会隐藏当前显示的内容面板，并显示对应的新面板，同时为当前激活的选项卡添加 `ui-tabs-active` 类名，以高亮样式标识当前选中的标签。选项卡的切换过程带有平滑的过渡效果，使操作反馈更加自然。通过选项卡组件的使用，上传区域的两种功能被清晰地组织起来，用户不会因为多个上传表单同时显示而感到困惑，页面布局也更加紧凑整洁。系统使用的选项卡组件如图 3.5 所示。

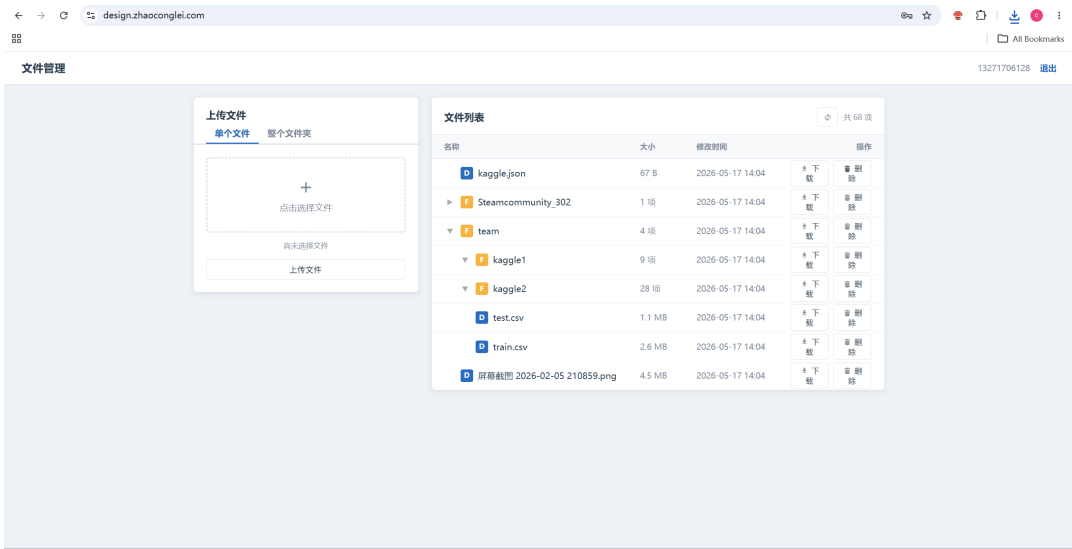


图 3.5: 选项卡组件

3.3.2 按钮组件

按钮组件（Button）是 jQuery UI 中最基础的交互组件之一，它能够将普通的 HTML 按钮（`<button>` 元素和 `<input type="submit">` 元素）美化为具有统一视觉风格的 UI 控件，并支持悬停、点击、禁用和聚焦等多种交互状态。

在本系统中，按钮组件被应用于页面中所有的操作按钮，包括上传区域中的”上传文件”按钮和”上传文件夹”按钮、文件管理区域中的”刷新”按钮，以及文件列表中的”下载”和”删除”按钮。页面加载时通过 `$("button").button()` 对所有按钮进行统一初始化，或对动态生成的文件列表按钮在渲染完成后单独调用 `.button()` 方法应用样式。

按钮组件提供了丰富的配置选项。对于刷新按钮，系统通过 `$("#refreshFiles").button({ icon` 为其添加了刷新图标并隐藏文字标签，使按钮更加简洁直观。在上传过程中，系统通过 `$btn.button("option", "disabled", true)` 将上传按钮设置为禁用状态，防止用户在传输过程中重复点击提交，同时按钮会自动变为灰色样式以表示不可操作状态。

上传完成或失败后, 再通过 `.button("option", "disabled", false)` 恢复按钮的可点击状态。

在样式定制方面, 系统通过 CSS 覆盖了按钮组件的默认外观。普通按钮使用白色背景和灰色边框, 悬停时边框变为主题蓝色并添加浅蓝色背景。通过自定义 `.primary-btn` 和 `.danger-btn` 类名, 对话框中的确定按钮使用蓝色主题色, 删除按钮使用红色警示色, 使用户能够通过颜色直观地区分不同性质的按钮操作。系统使用的按钮组件如图 3.6 所示。

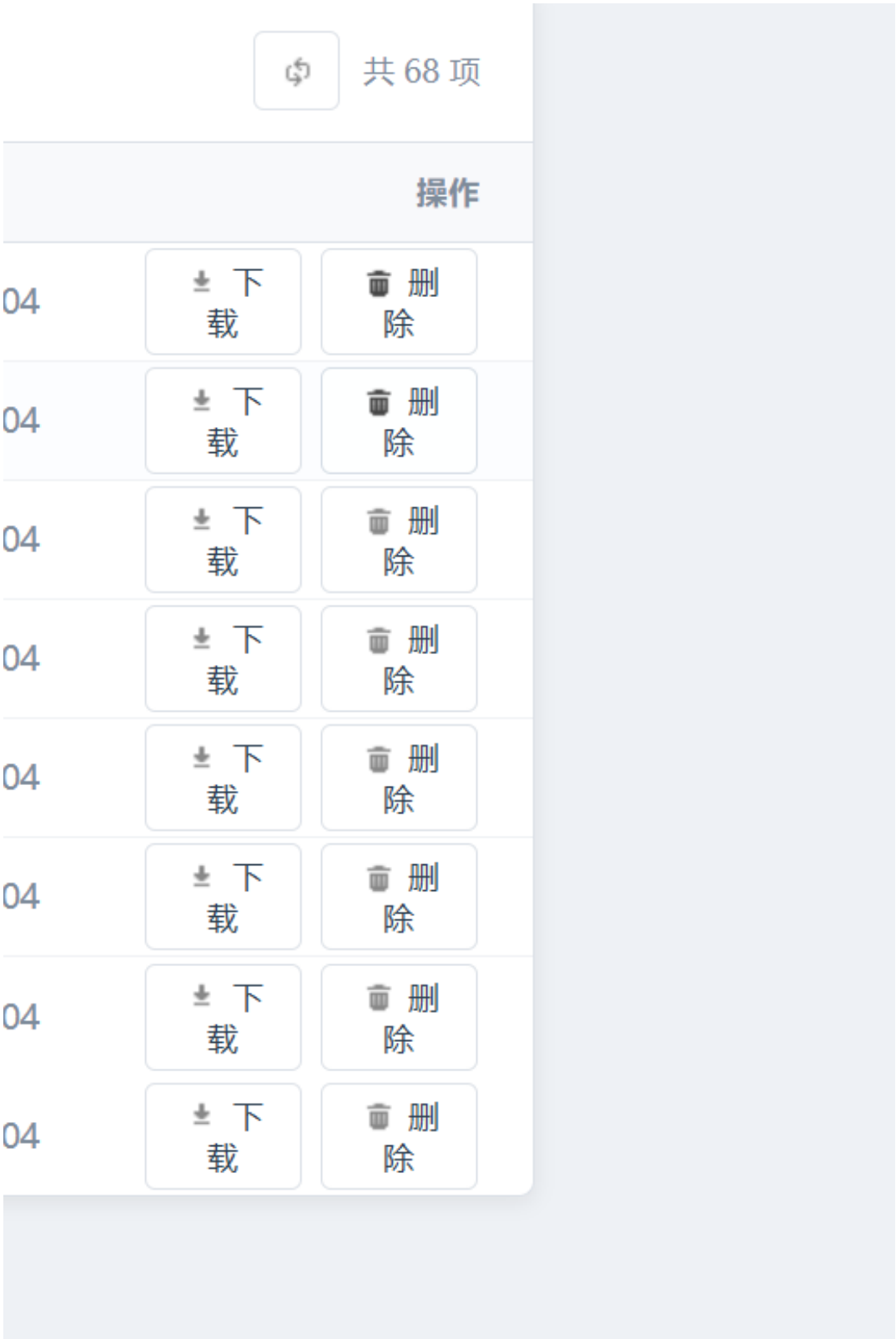


图 3.6: 按钮组件

3.3.3 进度条组件

进度条组件（Progressbar）是 jQuery UI 中用于显示任务进度的组件，它通过一个水平条形图直观地展示当前任务的完成比例。进度条通常用于文件上传、数据加载和批量处理等耗时操作的进度反馈场景。

在本系统中，进度条组件用于文件上传过程中的进度显示。进度条放置在文件上传区域的选项卡上方，初始状态为隐藏。当用户提交上传表单时，进度条通过 `$("#uploadProgress").progressbar()` 方法被初始化和显示，同时将进度值设为 0。

进度条的更新依赖于 XMLHttpRequest 的 `upload.onprogress` 事件。当浏览器向后端发送上传请求时，该事件会随着数据传输的进行被多次触发，每次触发时事件对象中都包含 `loaded`（已传输字节数）和 `total`（总字节数）两个属性。前端通过计算 `Math.round((evt.loaded / evt.total) * 100)` 得到当前的完成百分比，并调用 `$("#uploadProgress").progressbar("value", pct)` 实时更新进度条的显示状态。当上传完成时，进度条到达 100%，并在短暂停留后自动隐藏，准备下一次上传操作。

进度条组件的使用有效地解决了文件上传过程中用户无法判断传输状态的问题。用户可以通过进度条直观地了解上传的进展情况，避免了因等待不确定而产生的焦虑感，也减少了用户因误判上传状态而进行重复操作的可能性。系统使用的进度条组件如图 3.7 所示。



图 3.7: 进度条组件

3.3.4 对话框组件

对话框组件（Dialog）是 jQuery UI 中用于创建模式弹窗的组件，它可以显示提示信息、确认操作或表单内容，并支持拖拽移动、调整大小和键盘操作等功能。对话框通常以模态（Modal）形式出现，即对话框打开时页面背景内容不可操作，用户必须先处理对话框中的内容才能继续操作。

在本系统中，对话框组件被用于两个场景：消息提示和删除确认。

消息提示对话框用于在操作成功或失败时向用户反馈结果信息。系统通过 `$("#msgDialog").dialog("open")` 对消息对话框进行初始化，设置其为模态模式并默认关闭。需要显示消息时，调用 `$("#msgText").text(msg)` 设置提示文本，然后通过 `$("#msgDialog").dialog("open")` 打开对话框。对话框中包含一个“确定”按钮，用户点击后关闭对话框。消息提示对话框适用于上传成功、下载准备、删除完成和操作失败等多种场景，为用户提供了统一的反馈入口。

删除确认对话框用于在执行删除操作前请求用户二次确认。当用户点击删除按钮时，系统首先在对话框中显示将要删除的文件或文件夹名称，并提示“此操作不可恢复”等警示信息，然后通过 `$("#deleteDialog").dialog("open")` 打开确认对话框。对话框中包含“取消”和“删除”两个按钮，用户点击“取消”关闭对话框不执行任何操作，点击“删除”则向后端发送删除请求完成删除。这种确认机制有效地防止了用户因误操作而删除重要文件。

在样式方面，系统对对话框组件进行了定制，包括圆角边框、阴影效果、自定义关闭按钮样式和按钮组布局等，使对话框的整体风格与系统的主题配色协调一致。系统使用的删除确认对话框如图 3.8 所示。

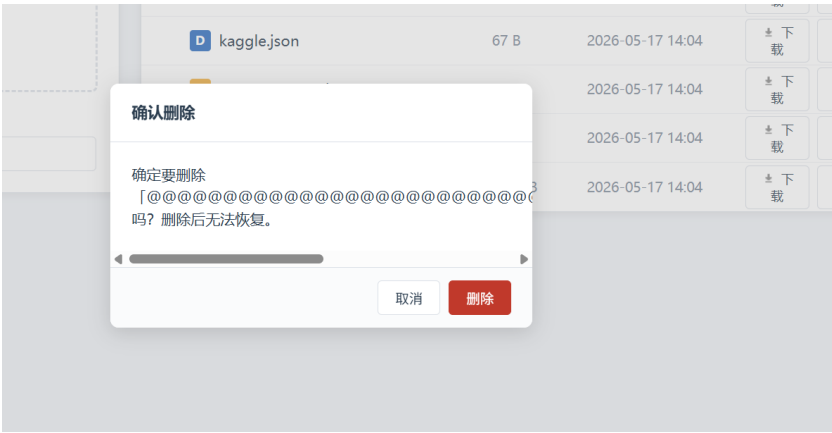


图 3.8: 删除确认对话框

3.4 文件上传功能实现

文件上传是文件管理系统的基础功能，用户通过该功能将本地计算机中的文件传输到服务器的存储目录中。本系统支持单文件上传和文件夹上传两种方式，分别对应不同的前端表单和后端路由。

3.4.1 单文件上传实现

单文件上传功能通过 `/upload` 路由实现。前端对应的 HTML 表单定义了一个 `<input type="file">` 文件选择控件，用户点击文件选择区域后打开系统对话框选取文件。选定文件后，JavaScript 通过 `change` 事件监听器捕获用户选择的文件信息并将其名称显示在页面上，供用户确认。

前端上传逻辑由 `uploadForm()` 函数封装。该函数接收 jQuery 表单对象、请求 URL 和成功提示信息三个参数，利用 `FormData` 对象将表单数据序列化后，通过 Ajax POST 请求发送至后端。上传开始前，函数将提交按钮设置为禁用状态并将按钮文字改为“上传中...”，同时显示进度条组件，防止用户重复提交。上传完成后，无论成功或失败，函数都将恢复按钮状态并隐藏进度条，确保下次上传时组件处于初始状态。

后端 `/upload` 路由的处理函数 `receive_file()` 通过 `request.files.get("upload_file")` 获取前端上传的文件对象。函数首先检查文件对象是否存在且文件名不为空，若文件无效则返回 400 状态码和错误提示信息。文件验证通过后，系统调用 `safe_upload_path()` 函数对文件路径进行安全规范化处理，确保上传路径合法且位于 `uploads` 目录范围内。路径验证通过后，使用 `os.makedirs()` 保证目标子目录存在，最后调用 `file.save()` 将文件保存至服务器文件系统。保存成功后，后端返回包含文件路径的 JSON 响应，前端接收到响应后调用 `showMessage()` 显示上传成功的提示信息，同时重置表单并刷新文件列表。

3.4.2 文件夹上传实现

文件夹上传功能通过 `/upload_folder` 路由实现，与单文件上传共用同一个上传区域，通过选项卡组件与单文件上传进行切换。

前端通过 HTML5 提供的 `webkitdirectory` 属性实现文件夹选择能力。当用户在文件夹上传标签页中点击文件选择区域时，系统文件对话框允许用户选取整个文件夹。选定后，`change` 事件监听器获取文件夹中的所有文件对象，并通过 `files`

0

`.webkitRelativePath` 属性获取各文件在文件夹中的相对路径。页面上会显示文件夹名称及内部文件总数，供用户确认。

表单提交时，前端的 `uploadForm()` 函数将包含所有文件的 `FormData` 对象发送到 `/upload_folder` 接口。后端 `upload_folder()` 处理函数通过 `request.files.getlist("upload_fo`

获取文件夹中的全部文件列表，随后遍历该列表逐个处理每个文件。对于每个文件，系统调用 `safe_upload_path()` 规范化其相对路径，在目标路径下逐级创建子目录，最后保存文件。由于每个文件在 `FormData` 中保留了其相对于根文件夹的路径信息，服务器端能够精确还原用户本地的目录结构。全部文件保存完成后，后端返回包含成功保存文件数量和文件列表的 JSON 响应。

值得注意的是，文件夹上传并非一次性传输整个文件夹，而是由浏览器在前端遍历文件夹内所有文件，为每个文件生成一个独立的表单项，批量提交给后端逐个保存。这种方式不需要特殊的传输协议支持，完全基于标准 HTML 表单和 HTTP 文件上传机制即可实现目录结构的保留。

3.4.3 上传进度显示

上传进度显示使用 jQuery UI 的 Progressbar 组件实现。进度条默认隐藏，在上传开始时显示并设置为不确定状态（indeterminate），以表示数据传输正在进行中。当使用基于 XMLHttpRequest 的 Ajax 上传时，浏览器会在数据传输过程中触发 `progress` 事件，事件对象包含 `loaded`（已传输字节数）和 `total`（总字节数）属性。前端代码计算当前进度百分比后，调用 `progressbar("value", pct)` 方法实时更新进度条。上传完成后进度条自动隐藏，恢复到初始状态等待下次使用。

3.5 文件下载功能实现

文件下载功能使用户能够从服务器获取已保存的文件或打包后的文件夹压缩包。系统为文件列表中的每个文件和文件夹均提供了下载按钮，用户点击后直接触发下载，无需额外确认步骤。

3.5.1 后端下载逻辑实现

文件下载功能统一由 `/download/<path:file_path>` 路由处理，其中 `<path:file_path>` 为 Flask 的动态路径参数，能够匹配包含多级子目录的路径。

处理函数 `download_file()` 首先调用 `safe_upload_path()` 对用户请求的路径进行安全校验，确保路径合法且不包含越权访问。路径验证通过后，函数检查目标路径在服务器文件系统中是否存在，若不存在则返回 404 状态码。路径存在的情况下，函数通过 `os.path.isdir()` 判断目标路径是文件还是文件夹，然后分别采用不同的处理方式。

对于单个文件的下载，系统直接使用 Flask 提供的 `send_file()` 函数，将目标文件作为附件响应返回给浏览器。`send_file()` 函数自动处理文件的 MIME 类型判定，并通过 `as_attachment=True` 参数使浏览器弹出文件保存对话框，而不是直接尝试在浏览器中打开文件。文件名通过 `download_name` 参数指定为原始文件名，确保用户下载后得到的文件名与服务器保存的文件名一致。

对于文件夹的下载，系统将整个文件夹递归地打包为一个 zip 压缩包后再提供给用户。打包逻辑封装在 `send_directory_zip()` 函数中。该函数首先创建一个 `BytesIO` 内存缓冲区用于存放压缩包数据，然后使用 Python 标准库中的 `zipfile.ZipFile` 以 `ZIP_DEFLATED` 压缩模式写入内存缓冲区。函数使用 `os.walk()` 递归遍历文件夹中的所有文件和子文件夹，对每个文件调用 `zipfile.write()` 将其添加到压缩包中，并设置 `arcname` 参数为相对于父目录的路径，从而在压缩包中保留完整的目录结构。压缩完成后，将内存缓冲区的指针移回起始位置，最后通过 `send_file()` 将内存中的压缩包数据以文件附件的形式发送给浏览器，文件名使用文件夹名称加上 `.zip` 后缀。

3.5.2 前端下载触发机制

前端的下载操作通过事件委托机制实现。文件列表中的下载按钮（`.dl-btn`）是在页面加载后由 JavaScript 动态渲染生成的，因此不能直接在按钮上绑定事件处理器。系统通过在文件列表容器（`#fileList`）上注册 `click` 事件监听器，利用事件冒泡机制捕获所有子元素上发生的点击事件。当用户点击任意下载按钮时，事件处理器从按钮的 `data-path` 属性中读取文件或文件夹的存储路径，然后通过 `window.location.href` 将浏览器地址跳转到下载接口的 URL。这种方式利用了浏览器的原生文件下载机制，无需额外处理响应数据，浏览器会自动根据服务器返回的 `Content-Disposition` 响应头触发文件保存操作。路径中的特殊字符通过 `encodeURIComponent()` 进行编码，确保包含中文或空格的文件名能正确地作为 URL 参数传递。

3.6 文件删除功能实现

文件删除功能用于清理服务器上不再需要的文件和文件夹。考虑到删除操作的不可逆性，系统在删除前增加了确认环节，防止用户误操作。

3.6.1 前端删除流程

前端删除操作由确认阶段和执行阶段两个步骤组成。当用户点击文件列表中的删除按钮时,首先触发确认对话框的显示。事件处理器通过 `data-path` 和 `data-name` 属性获取待删除目标的路径和名称,将名称信息嵌入确认提示文本中,然后通过 `$("#deleteDialog").dialog("open")` 打开 jQuery UI 的对话框组件。对话框中包含“取消”和“删除”两个按钮,分别对应关闭对话框和发起删除请求两种操作。

用户点击“删除”按钮确认后,系统执行 `removePath()` 函数。该函数通过 Ajax POST 请求向 `/delete/<path>` 接口发送删除指令。请求路径同样经过 `encodedPath()` 函数进行 URI 编码处理,确保路径中的非 ASCII 字符和特殊字符不会导致请求失败。删除请求发送后,函数等待后端返回处理结果:成功时显示操作成功的提示信息并自动刷新文件列表;失败时显示错误提示信息,告知用户删除操作未能完成。整个删除过程中,用户不需要手动刷新页面,文件列表会在删除成功后自动更新。

3.6.2 后端删除逻辑

后端 `/delete/<path:file_path>` 路由的处理函数 `delete_file()` 首先通过 `safe_upload_path()` 对请求路径进行安全校验。路径验证通过后,函数检查目标路径在服务器文件系统中是否存在,若文件或文件夹不存在则返回 404 状态码和对应的错误提示。

路径存在的情况下,函数通过 `os.path.isdir()` 判断目标是文件还是文件夹,然后分别处理。对于单一文件,使用 `os.remove()` 将其从文件系统中移除。对于文件夹,使用 `shutil.rmtree()` 递归删除文件夹内的所有文件和子文件夹。这种递归删除方式确保了即使文件夹内部包含多层嵌套的目录结构和大量文件,也能被完整地清理干净,不会留下残留的临时文件或空目录。

删除操作完成后,后端返回 JSON 格式的成功响应,包含删除操作的描述信息和被删除目标的路径。前端接收到成功响应后,通过 `showMessage()` 显示操作结果,并调用 `loadFilesList()` 刷新文件列表,使用户可以在页面上立即看到删除操作生效后的最新文件状态。

3.7 异常处理与安全设计

系统的异常处理和安全性设计覆盖了文件路径安全防护、用户访问控制、操作异常处理等多个方面,是保证系统稳定运行和数据安全的重要保障。

3.7.1 文件路径安全防护

文件路径安全是本系统的核心安全问题之一。由于系统涉及文件的上传、下载和删除操作，所有功能都需要根据用户提供的路径信息访问服务器的文件系统。如果不对用户路径进行严格的校验，攻击者可能通过路径穿越攻击，利用 `../` 等相对路径标记访问 `uploads` 目录以外的系统文件，造成信息泄露甚至系统破坏。

针对这一问题，系统实现了两层路径安全防护机制。第一层是路径规范化函数 `normalize_upload_path()`。该函数首先将路径中的反斜杠统一替换为正斜杠并去除首尾的斜杠，然后将路径按斜杠分割为各部分，过滤掉空字符串和当前目录标记 (`.`)。随后检查路径各部分中是否包含上级目录标记 (`..`)，如果包含则直接终止请求并返回 400 错误。这一处理可以有效防止用户通过 `../../../etc/passwd` 等恶意路径绕过目录限制。

第二层是路径安全校验函数 `safe_upload_path()`。该函数在 `normalize_upload_path()` 的基础上，将规范化后的路径各部分与 `uploads` 目录的绝对路径拼接，然后使用 `os.path.abspath()` 获取最终的绝对路径。关键的安全校验在于：函数使用 `os.path.commonpath()` 比较目标路径的绝对路径与 `uploads` 目录的绝对路径的公共前缀，如果公共前缀不等于 `uploads` 目录路径，说明目标路径位于 `uploads` 目录之外，则返回 400 错误。这种校验方式即使攻击者通过编码绕过或其他方式使 `..` 标记逃脱了第一层过滤，也会在第二层被检测并拦截。

3.7.2 用户访问控制

系统通过 Flask 的 `session` 机制实现基本的用户访问控制。当用户成功登录后，后端将用户名写入 `session` 对象，后续每个需要认证的请求都可以通过检查 `session` 中是否包含 `"username"` 键来判断用户的登录状态。

在路由层面，主页路由 `index()` 在处理请求前首先检查 `session` 中是否存在用户信息，如果用户未登录则自动重定向到注册页面。这一机制确保了未登录用户无法访问文件管理主页和操作文件资源。而对于上传、下载、删除和文件列表等接口，当前版本在路由层面默认仅在登录后才能通过页面进行操作，因为前端页面本身已经处于登录保护之下。

用户密码以明文形式存储在 `user.json` 文件中。这种方式虽然不适用于生产环境，但对于本课程设计项目而言，实现简单且足以验证用户认证的完整流程。用户注册时，系统检查用户名是否已存在，避免重复注册。用户登录时，系统验证用户名和密码与保存的信息是否完全匹配，匹配成功后才允许登录。

3.7.3 操作异常处理

系统在文件上传、下载和删除的各个环节均实现了异常处理逻辑，确保在异常情况下系统能够给出友好的错误反馈而不是直接崩溃。

在文件上传方面，系统在前端和后端两个层面进行验证。前端在上传表单提交时检查用户是否已选择文件，若未选择则直接提示用户“请选择要上传的文件”而不发起网络请求；后端同样在接收文件时检查文件对象是否为空，对空文件返回 400 错误。上传过程中如果网络中断或服务器返回错误响应，前端的 `Ajax fail` 回调会解析错误信息并通过消息对话框展示给用户。

在文件下载方面，后端在接收到下载请求后首先验证目标文件是否存在。如果用户请求的文件或文件夹已被删除或路径不合法，后端返回 404 状态码，前端在下载失败时不会有明确的错误弹窗提示（因为下载通过浏览器直接跳转实现），但浏览器会显示标准的无法访问页面。

在文件删除方面，系统同样在删除前检查目标是否存在。如果删除操作执行时目标已被其他用户或操作提前删除，后端返回 404 错误和提示信息，前端在接收到错误响应后通过消息对话框告知用户“文件不存在”。对于服务器的其他未预期错误，Flask 框架的默认错误处理机制会捕获异常并返回 500 状态码，前端 `Ajax` 调用会触发 `fail` 回调并显示通用的“删除失败”提示信息，保证用户能够感知到操作异常。

第四章 系统测试

4.1 测试环境

4.1.1 硬件环境

系统部署于一台位于中国台湾的云服务器上，通过 Cloudflare CDN 进行内容分发与访问加速。服务器的主要硬件配置如下：处理器为 2 核心 x86_64 架构 CPU，主频不低于 2.0 GHz；内存容量为 2 GB；磁盘为 40 GB 的 SSD 云硬盘，用于存放系统文件、用户上传数据以及日志文件；网络带宽为 100 Mbps，具备独立的公网 IP 地址。该配置能够满足中小规模的并发访问需求，支持多用户同时进行文件上传、下载和删除操作。

4.1.2 软件环境

系统软件环境的配置围绕后端服务和前端访问两个方面展开。后端运行在 Linux 操作系统上（内核版本为 Linux 5.x 或以上），使用 Python 3 作为运行时环境。Web 服务采用 Flask 开发服务器或 Gunicorn WSGI 应用服务器运行，前端通过 Nginx 作为反向代理将外部请求转发至后端应用，并利用 Cloudflare CDN 提供 HTTPS 加密传输和缓存加速服务。

在依赖库方面，系统仅使用 Flask 框架及其内置组件，无需额外的第三方 Python 扩展包。前端页面引用 jQuery 和 jQuery UI 1.13.3 作为交互组件库，样式文件为系统自定义的 CSS，字体与图标使用操作系统默认字体与 Unicode 字符实现。

在域名与网络安全方面，系统通过域名 design.zhaoconglei.com 对外提供服务，使用 Cloudflare 提供的 SSL/TLS 证书加密所有请求。经过部署和配置后，用户可以通过浏览器直接访问该域名进入系统。具体的软件环境版本信息如表 4.1 所示。

表 4.1: 软件环境配置表

项目	版本/名称
操作系统	Linux（内核 5.x 或以上）
Web 服务器	Nginx / Cloudflare CDN
运行时环境	Python 3
Web 框架	Flask 3.x
前端组件库	jQuery 3.x / jQuery UI 1.13.3
数据库	文件系统（File System）
域名	design.zhaoconglei.com
传输加密	HTTPS（Cloudflare SSL/TLS）

4.2 页面显示测试

页面显示测试的目的是验证系统各页面在不同操作场景下能否正确加载和显示。测试覆盖了用户注册页面、登录页面以及文件管理主页面三个核心页面。

在注册页面测试中，用户通过浏览器访问系统域名后被自动重定向至注册页面。页面显示了正确的标题”注册”和描述信息，用户名和密码输入框正常显示，注册按钮样式正确。在登录页面测试中，已注册用户能够看到登录表单，输入框和登录按钮样式与注册页面保持一致，页面整体风格统一。

在文件管理主页面测试中，用户登录后进入系统主页。页面按照设计分为左右两栏布局：左侧为文件上传区域，右侧为文件管理区域。上传区域中，选项卡组件将单文件上传和文件夹上传分别放置在两个标签页中，切换正常；进度条组件默认隐藏；文件选择区域样式正确。管理区域中，顶部显示”文件列表”标题和刷新按钮，右侧显示文件总数统计，表头依次为名称、大小和修改时间三列。各页面的实际显示效果如图 4.1 所示。

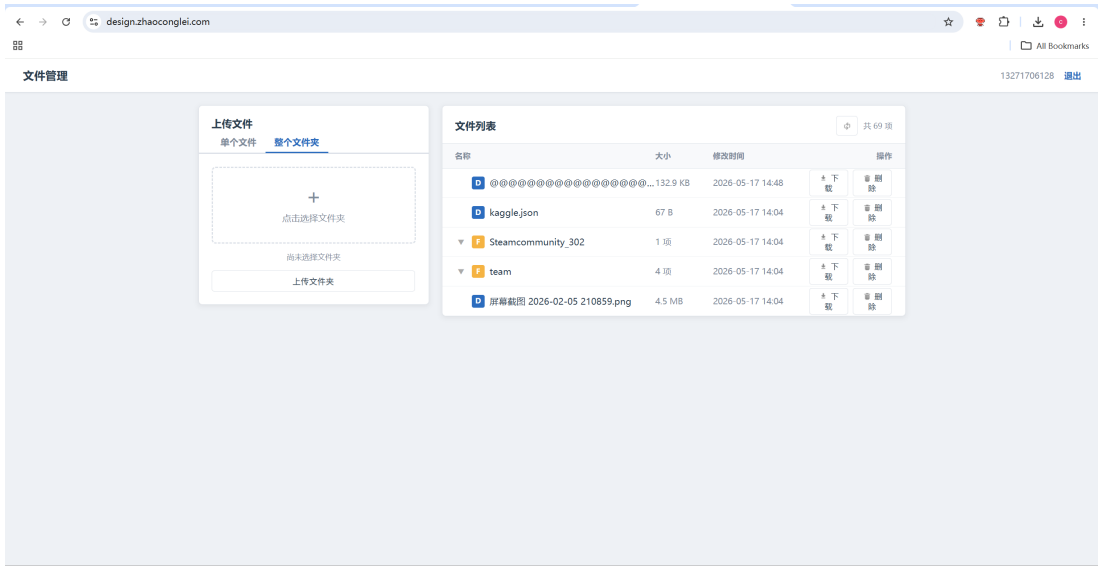


图 4.1: 页面显示效果

经测试，系统各页面均能正常加载，页面布局完整，CSS 样式正确应用，jQuery UI 组件的初始化状态与预期一致。

4.3 文件上传测试

文件上传测试的目的是验证系统能否正确接收并保存用户上传的文件，包括单文件上传和文件夹上传两种方式。

在单文件上传测试中，用户通过上传区域的”单个文件”标签页选择本地文件。测试选了一个文本文件（test.txt，大小 1.2 KB）和一个图片文件（screenshot.png，大小 256 KB）分别进行上传。选择文件后，页面上正确显示了已选择的文件名，点击”上传文件”按钮后，进度条显示上传进度，上传完成后弹出”文件上传成功”的提示信息。刷新文件列表后，已上传的文件正确显示在管理区域中，文件名、文件大小和修改时间信息均与实际文件一致。

在文件夹上传测试中，用户切换到”整个文件夹”标签页，选择了一个包含多个子文件夹和文件的测试目录。选定文件夹后，页面正确显示了文件夹名称及其包含的文件总数。点击”上传文件夹”按钮后，系统逐文件上传并显示进度。上传完成后，在文件管理区域中可以看到文件夹以目录结构的形式展示，展开文件夹后其内部的子文件夹和文件层级关系与本地原始目录完全一致。

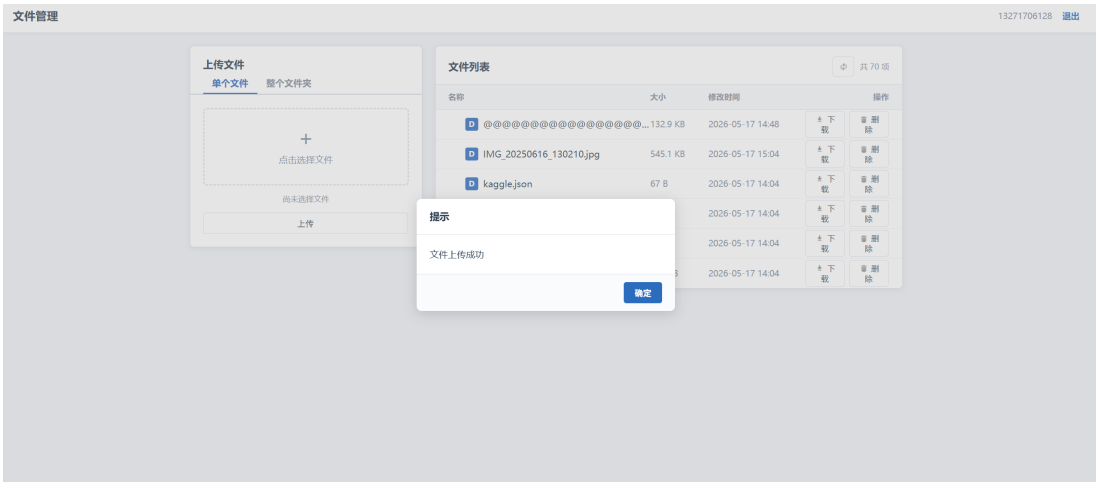


图 4.2: 文件上传功能测试

经测试，系统能够正确接收并保存用户上传的文件，单文件上传和文件夹上传功能均正常工作，上传后的文件目录结构正确，文件数据完整无损坏。

4.4 文件下载测试

文件下载测试的目的是验证系统能否正确地将服务器上的文件传输给用户，包括单个文件下载和文件夹打包下载两种场景。

在单个文件下载测试中，用户在文件管理区域找到已上传的文件，点击对应行右侧的” 下载” 按钮。浏览器正常弹出文件保存对话框，保存后的文件与上传前的原始文件内容一致，文件名正确。测试覆盖了文本文件、图片文件和 PDF 文件三种常见格式，所有文件均能正常下载且内容完整。

在文件夹下载测试中，用户点击文件夹行对应的” 下载” 按钮。系统自动将文件夹内的所有文件和子文件夹打包为一个 zip 压缩包，浏览器弹出文件保存对话框，保存的压缩包名为文件夹名称加上.zip 后缀。下载完成后，在本地解压该压缩包，内部的目录结构和文件内容与服务器上的原始目录完全一致，所有子文件夹的层级关系均被保留。

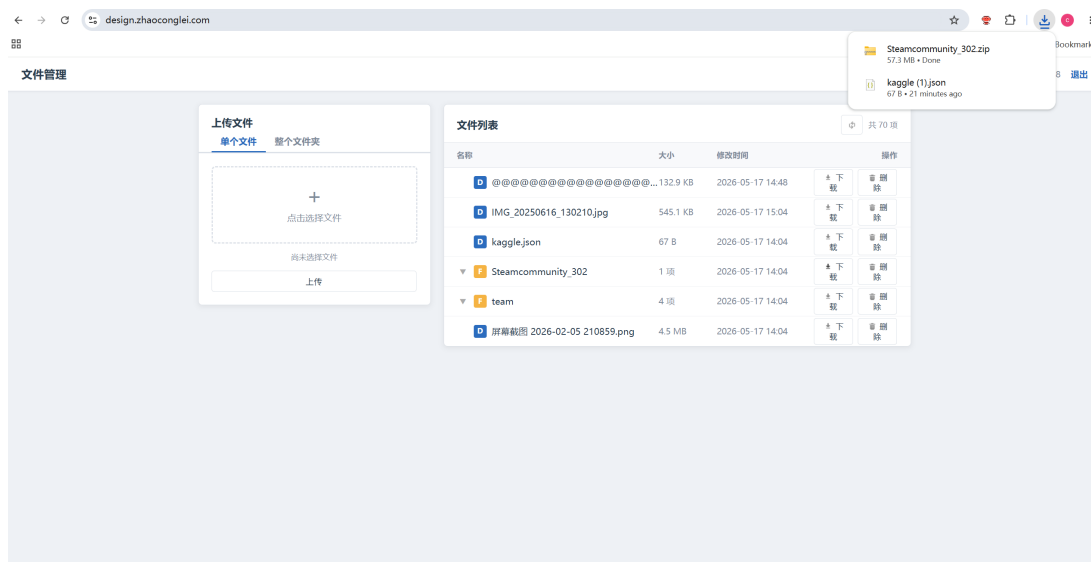


图 4.3: 文件下载功能测试

经测试，系统的文件下载功能运行正常。单个文件下载直接返回原始文件，文件夹下载自动打包为 zip 压缩包且目录结构完整。

4.5 文件删除测试

文件删除测试的目的是验证系统能否正确地从服务器上移除文件或文件夹，以及删除确认机制是否有效。

在文件删除测试中，用户首先在文件管理区域中点击某个文件行右侧的”删除”按钮。此时系统弹出删除确认对话框，对话框中正确显示了被删除文件的名称和警示提示文字。对话框包含”取消”和”删除”两个按钮。点击”取消”按钮后，对话框关闭，文件未被删除，文件列表保持不变。再次点击同一文件的”删除”按钮，然后在确认对话框中点击”删除”按钮，系统执行删除操作并弹出”文件删除成功”的提示信息，文件列表自动刷新，被删除的文件不再显示。

在文件夹删除测试中，用户对一个包含多个子文件和子文件夹的目录执行相同的删除操作。系统弹出确认对话框显示文件夹名称，确认删除后，文件夹及其内部的所有内容被完整移除，文件列表即时更新。测试还验证了删除不存在的文件或重复点击删除按钮等边界情况，系统均能返回正确的提示信息。

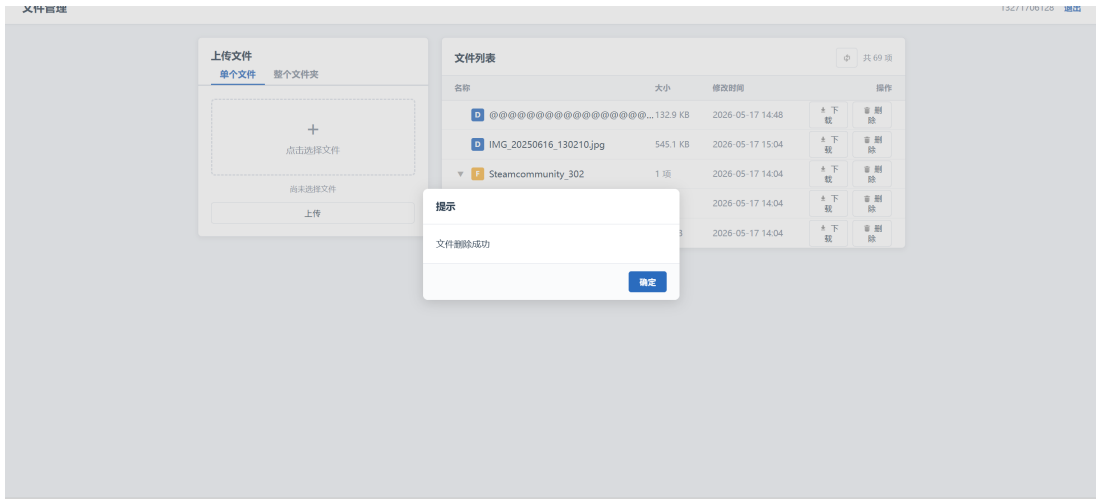


图 4.4: 文件删除功能测试

经测试，系统的文件删除功能运行正常。删除确认机制有效防止了误操作，删除操作能够正确地移除文件或整个文件夹，文件列表能够及时更新反映删除结果。

4.6 测试小结

本章对系统的页面显示、文件上传、文件下载和文件删除四个核心功能进行了全面的测试。测试结果如表 4.2 所示。

表 4.2: 测试结果汇总表

测试项目	测试结果	说明
页面显示测试	通过	各页面加载正常，布局和样式正确
单文件上传测试	通过	支持多种文件类型，内容完整
文件夹上传测试	通过	目录结构完整保留
单个文件下载测试	通过	下载文件与源文件一致
文件夹下载测试	通过	zip 打包正确，目录结构完整
文件删除测试	通过	确认机制有效，删除结果正确
文件夹删除测试	通过	递归删除完整，列表及时更新

从测试结果来看，系统实现了课程设计要求的所有核心功能。页面能够正确加载和显示，用户可以通过浏览器完成文件上传、下载和删除等基本操作，各项功能的实际表现与设计预期一致。系统的用户界面简洁清晰，操作流程直观，jQuery UI 组件的交互效果正常。在测试过程中未发现功能性的缺陷和错误，系统运行稳定，能够满足

基本的文件管理需求。

第五章 总结与展望

5.1 课程设计总结

本次课程设计完成了一个基于 Web 的文件上传、下载与删除管理系统的设计与实现。系统采用 Flask 框架作为后端服务，使用 HTML、CSS 和 JavaScript 构建前端页面，并结合 jQuery UI 组件库对页面交互进行增强，实现了文件上传、文件下载和文件删除三个核心功能。

在系统设计阶段，本文首先对文件管理系统的需求进行了详细分析，明确了单文件上传、文件夹上传、文件下载、文件夹打包下载、文件删除和文件夹删除等功能需求，并对用户注册登录、文件列表刷新、路径安全防护和交互增强等拓展功能进行了规划。在需求分析的基础上，本文对系统的整体架构进行了设计，采用浏览器/服务器（B/S）架构，将系统划分为前端展示层、后端业务层和数据存储层三个层次，各层职责清晰、相互独立。

在系统实现阶段，本文分别从前端界面和后端逻辑两个方面对系统进行了详细实现。前端使用 HTML 和 CSS 设计了文件上传区域和文件管理区域两个核心页面区域，利用 jQuery UI 的选项卡组件组织单文件上传和文件夹上传两种方式，使用按钮组件美化操作按钮，使用进度条组件显示上传进度，使用对话框组件展示提示信息和删除确认。后端使用 Flask 框架定义了文件上传、文件夹上传、文件下载、文件删除、文件列表获取以及用户注册登录等路由接口，实现了文件保存、文件读取、文件夹递归压缩、文件删除和路径安全校验等核心业务逻辑。

在系统测试阶段，本文对系统的页面显示、文件上传、文件下载和文件删除四个主要功能进行了全面的测试。测试结果表明，系统各项功能均能正常运行，页面显示正确，文件上传和下载的数据完整无误，文件删除操作准确可靠，用户界面的交互效果良好。系统整体运行稳定，达到了课程设计的预期目标。

通过本次课程设计，加深了对 Web 应用前后端交互流程的理解。前端方面，掌握了 HTML 页面结构组织、CSS 布局控制和 jQuery UI 组件使用方法；后端方面，熟

练运用了 Flask 框架的路由机制和请求处理流程，学习了文件系统操作、路径安全处理和服务器端文件流传输等技术。这些知识和经验对于后续学习更复杂的 Web 应用开发具有重要的实践意义。

5.2 系统不足

尽管本系统已经实现了课程设计要求的核心功能，但在以下几个方面仍存在不足和可以改进之处。

第一，用户数据以明文形式存储在 JSON 文件中，缺乏加密保护。当前系统的用户注册信息直接以明文密码写入 user.json 文件，虽然实现简单，但在安全性上存在隐患。在实际生产环境中，用户密码应当经过哈希加密处理后再进行存储，防止因数据泄露导致用户账号安全问题。

第二，系统缺少用户权限管理机制。当前所有已注册用户在登录后都可以查看和操作 uploads 目录下的所有文件，不同用户之间的文件没有实现隔离。这意味着一个用户上传的文件可以被其他用户查看、下载甚至删除，在多用户场景下存在数据安全和隐私保护方面的不足。

第三，文件存储完全依赖服务器文件系统，缺乏数据库支持。当前系统将文件直接保存在服务器的 uploads 目录中，文件元数据（如上传时间、上传者、文件类型等）无法被有效记录和查询。随着文件数量的增加，纯文件系统的方式在文件检索、批量管理和数据维护方面会变得越发困难。

第四，系统缺少文件预览功能。用户在上传文件后，无法直接在浏览器中预览文件内容。对于图片、文档和文本文件等常见格式，文件预览功能可以显著提升用户体验，让用户在不下载文件的情况下快速了解文件内容。

第五，前端页面缺少响应式设计的完善适配。当前页面的布局主要针对桌面端屏幕进行设计，在移动设备或小尺寸屏幕上的显示效果还有待优化。对于部分较窄的屏幕，文件列表可能会出现横向滚动或文字溢出的情况。

5.3 改进方向

针对上述不足，系统可以从以下几个方向进行改进和扩展。

在用户管理方面，可以引入密码哈希加密机制，使用 bcrypt 或 Argon2 等成熟的密码哈希算法对用户密码进行加密存储，提高系统的安全性。同时，可以建立基于角

色的权限管理模型（RBAC），为不同用户分配不同的文件访问范围和操作权限，实现用户文件隔离和多级权限控制。

在数据存储方面，可以引入关系型数据库（如 SQLite、MySQL）或 NoSQL 数据库（如 MongoDB）来替代纯 JSON 文件存储用户信息，并使用数据库记录文件的上传时间、上传者、文件类型、文件大小等元数据，方便进行文件检索、统计分析和批量管理。文件存储也可以考虑使用云存储服务（如阿里云 OSS、腾讯云 COS）来替代服务器本地磁盘，以提高存储容量和可靠性。

在功能扩展方面，可以增加文件预览功能，对于图片、PDF、文本文件等常见格式直接在浏览器中展示内容，对于音视频文件提供在线播放能力。还可以增加文件搜索功能，支持按文件名、文件类型、上传时间等条件进行检索，方便用户在大量文件中快速定位目标文件。此外，可以引入文件分享功能，允许用户生成文件的临时分享链接，方便系统外部人员通过链接访问特定文件。

在前端体验方面，可以对页面进行响应式改造，使用 CSS 媒体查询和弹性布局使页面在不同屏幕尺寸下均能正常显示和操作。同时，可以引入前端框架（如 Vue.js、React）来替代传统的 jQuery 开发方式，实现更高效的页面状态管理和组件复用。上传体验方面可以支持拖拽上传和批量上传，提升文件传输的便捷性。

在部署与运维方面，可以使用 Docker 容器化技术将系统打包为标准镜像，简化部署流程和环境配置。可以引入 CI/CD 持续集成和持续部署流水线，实现代码提交后自动测试和自动部署。还可以增加日志记录和监控告警功能，记录系统的运行状态和操作日志，便于问题排查和系统维护。